

Importing Libraries

```
In [1]: import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

Step 1: Understand the Dataset

```
In [2]: df=pd.read_csv('Service_request.csv')
df.head() # Visualize dataset
```

C:\Users\Priyanka Kesarkar\AppData\Local\Temp\ipykernel_788\3704723030.py:1: DtypeWarning: Columns (48,49) have mixed types. Specify dtype option on import or set low_memory=False.

```
df=pd.read_csv('Service_request.csv')
```

Out[2]:

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Descriptor	Location
0	32310363	12/31/2015 11:59:45 PM	01/01/2016 12:55:15 AM	NYPD	New York City Police Department	Noise - Street/Sidewalk	Loud Music/Party	Street
1	32309934	12/31/2015 11:59:44 PM	01/01/2016 01:26:57 AM	NYPD	New York City Police Department	Blocked Driveway	No Access	Street
2	32309159	12/31/2015 11:59:29 PM	01/01/2016 04:51:03 AM	NYPD	New York City Police Department	Blocked Driveway	No Access	Street
3	32305098	12/31/2015 11:57:46 PM	01/01/2016 07:43:13 AM	NYPD	New York City Police Department	Illegal Parking	Commercial Overnight Parking	Street
4	32306529	12/31/2015 11:56:58 PM	01/01/2016 03:24:42 AM	NYPD	New York City Police Department	Illegal Parking	Blocked Sidewalk	Street

5 rows × 53 columns



```
In [3]: # Print columns
df.columns
```

```
Out[3]: Index(['Unique Key', 'Created Date', 'Closed Date', 'Agency', 'Agency Name',  
             'Complaint Type', 'Descriptor', 'Location Type', 'Incident Zip',  
             'Incident Address', 'Street Name', 'Cross Street 1', 'Cross Street 2',  
             'Intersection Street 1', 'Intersection Street 2', 'Address Type',  
             'City', 'Landmark', 'Facility Type', 'Status', 'Due Date',  
             'Resolution Description', 'Resolution Action Updated Date',  
             'Community Board', 'Borough', 'X Coordinate (State Plane)',  
             'Y Coordinate (State Plane)', 'Park Facility Name', 'Park Borough',  
             'School Name', 'School Number', 'School Region', 'School Code',  
             'School Phone Number', 'School Address', 'School City', 'School State',  
             'School Zip', 'School Not Found', 'School or Citywide Complaint',  
             'Vehicle Type', 'Taxi Company Borough', 'Taxi Pick Up Location',  
             'Bridge Highway Name', 'Bridge Highway Direction', 'Road Ramp',  
             'Bridge Highway Segment', 'Garage Lot Name', 'Ferry Direction',  
             'Ferry Terminal Name', 'Latitude', 'Longitude', 'Location'],  
            dtype='object')
```

```
In [4]: # Shape of dataset  
df.shape
```

```
Out[4]: (364558, 53)
```

```
In [5]: # Check Null Values  
df.isnull().sum()
```

```

Out[5]: Unique Key                0
        Created Date              0
        Closed Date               2381
        Agency                   0
        Agency Name               0
        Complaint Type            0
        Descriptor                6501
        Location Type             133
        Incident Zip              2998
        Incident Address          51699
        Street Name               51699
        Cross Street 1            57188
        Cross Street 2            57805
        Intersection Street 1     313438
        Intersection Street 2     314046
        Address Type              3252
        City                      2997
        Landmark                  364183
        Facility Type             2389
        Status                    0
        Due Date                  3
        Resolution Description     0
        Resolution Action Updated Date 2402
        Community Board           0
        Borough                   0
        X Coordinate (State Plane) 4030
        Y Coordinate (State Plane) 4030
        Park Facility Name        0
        Park Borough              0
        School Name               0
        School Number             0
        School Region             1
        School Code               1
        School Phone Number       0
        School Address            0
        School City               0
        School State              0
        School Zip                1
        School Not Found          0
        School or Citywide Complaint 364558
        Vehicle Type              364558
        Taxi Company Borough      364558
        Taxi Pick Up Location     364558
        Bridge Highway Name       364261
        Bridge Highway Direction  364261
        Road Ramp                 364296
        Bridge Highway Segment    364296
        Garage Lot Name           364558
        Ferry Direction           364557
        Ferry Terminal Name       364556
        Latitude                  4030
        Longitude                 4030
        Location                  4030
        dtype: int64

```

```
In [6]: df.duplicated().sum()
```

```
Out[6]: np.int64(0)
```

```
In [7]: # Remove Agency Name Column  
df.drop('Agency Name', axis = 1, inplace = True)
```

```
In [8]: df['Complaint Type'].isnull().sum()
```

```
Out[8]: np.int64(0)
```

```
In [9]: # Filling NA value of descriptor  
df['Descriptor'].isnull().sum()
```

```
Out[9]: np.int64(6501)
```

```
In [10]: df['Descriptor'].describe()
```

```
Out[10]: count      358057  
         unique        45  
         top      No Access  
         freq      75888  
         Name: Descriptor, dtype: object
```

```
In [11]: df.Descriptor.value_counts()
```

```
Out[11]: Descriptor
No Access 75888
Loud Music/Party 69708
Posted Parking Sign Violation 27200
Partial Access 24993
Loud Talking 23810
With License Plate 21661
Blocked Hydrant 19989
Commercial Overnight Parking 14997
Blocked Sidewalk 13860
Car/Truck Music 11940
Double Parked Blocking Traffic 7164
Engine Idling 5323
Double Parked Blocking Vehicle 5167
Neglected 5131
Banging/Pounding 4959
Car/Truck Horn 4478
Congestion/Gridlock 3237
Other (complaint details) 2489
In Prohibited Area 2206
Overnight Commercial Storage 2102
Unlicensed 1986
Unauthorized Bus Layover 1618
Truck Route Violation 1125
Tortured 1041
In Public 971
No Shelter 886
Chained 695
Vehicle 602
Detached Trailer 582
Underage - Licensed Est 345
Chronic Stoplight Violation 339
Chronic Speeding 306
In Car 299
Playing in Unsuitable Place 272
Drag Racing 191
Police Report Requested 122
Loud Television 114
After Hours - Licensed Est 93
Building 79
Nuisance/Truant 43
Police Report Not Requested 35
Language Access Complaint 8
Homeless Issue 1
Disruptive Passenger 1
Animal Waste 1
Name: count, dtype: int64
```

```
In [12]: df['Descriptor'].fillna('No Description', inplace=True)
```

```
In [13]: df['Descriptor'].isnull().sum()
```

```
Out[13]: np.int64(0)
```

```
In [14]: df.Descriptor.value_counts()
```

```
Out[14]: Descriptor
No Access 75888
Loud Music/Party 69708
Posted Parking Sign Violation 27200
Partial Access 24993
Loud Talking 23810
With License Plate 21661
Blocked Hydrant 19989
Commercial Overnight Parking 14997
Blocked Sidewalk 13860
Car/Truck Music 11940
Double Parked Blocking Traffic 7164
No Description 6501
Engine Idling 5323
Double Parked Blocking Vehicle 5167
Neglected 5131
Banging/Pounding 4959
Car/Truck Horn 4478
Congestion/Gridlock 3237
Other (complaint details) 2489
In Prohibited Area 2206
Overnight Commercial Storage 2102
Unlicensed 1986
Unauthorized Bus Layover 1618
Truck Route Violation 1125
Tortured 1041
In Public 971
No Shelter 886
Chained 695
Vehicle 602
Detached Trailer 582
Underage - Licensed Est 345
Chronic Stoplight Violation 339
Chronic Speeding 306
In Car 299
Playing in Unsuitable Place 272
Drag Racing 191
Police Report Requested 122
Loud Television 114
After Hours - Licensed Est 93
Building 79
Nuisance/Truant 43
Police Report Not Requested 35
Language Access Complaint 8
Homeless Issue 1
Disruptive Passenger 1
Animal Waste 1
Name: count, dtype: int64
```

```
In [15]: # Filling NA Value of Location Type Column
df['Location Type'].isnull().sum()
```

```
Out[15]: np.int64(133)
```

```
In [16]: df['Location Type'].describe()
```

```
Out[16]: count          364425  
         unique           18  
         top      Street/Sidewalk  
         freq          301372  
         Name: Location Type, dtype: object
```

```
In [17]: df['Location Type'].value_counts()
```

```
Out[17]: Location Type  
Street/Sidewalk          301372  
Store/Commercial        25217  
Club/Bar/Restaurant     21527  
Residential Building/House  8869  
Park/Playground         4871  
House of Worship        1068  
Residential Building      533  
Highway                  265  
House and Store           245  
Parking Lot              167  
Commercial               108  
Vacant Lot                95  
Roadway Tunnel           43  
Subway Station           40  
Bridge                   2  
Terminal                 1  
Ferry                   1  
Park                    1  
Name: count, dtype: int64
```

```
In [18]: df['Location Type'].fillna('Unknown', inplace = True)
```

```
In [19]: df['Location Type'].isnull().sum()
```

```
Out[19]: np.int64(0)
```

```
In [20]: df['Location Type'].value_counts()
```

```
Out[20]: Location Type
          Street/Sidewalk      301372
          Store/Commercial    25217
          Club/Bar/Restaurant  21527
          Residential Building  8869
          Park/Playground      4871
          House of Worship     1068
          Residential Building   533
          Highway              265
          House and Store       245
          Parking Lot          167
          Unknown              133
          Commercial           108
          Vacant Lot           95
          Roadway Tunnel        43
          Subway Station        40
          Bridge                2
          Terminal              1
          Ferry                 1
          Park                  1
          Name: count, dtype: int64
```

```
In [21]: # Filling NA Value of Incident Zip Column
df['Incident Zip'].isnull().sum()
```

```
Out[21]: np.int64(2998)
```

```
In [22]: df['Incident Zip'].value_counts().head()
```

```
Out[22]: Incident Zip
11385.0    6396
11368.0    5384
11234.0    5199
11211.0    4854
11377.0    4525
          Name: count, dtype: int64
```

```
In [23]: df['Incident Zip'].describe()
```

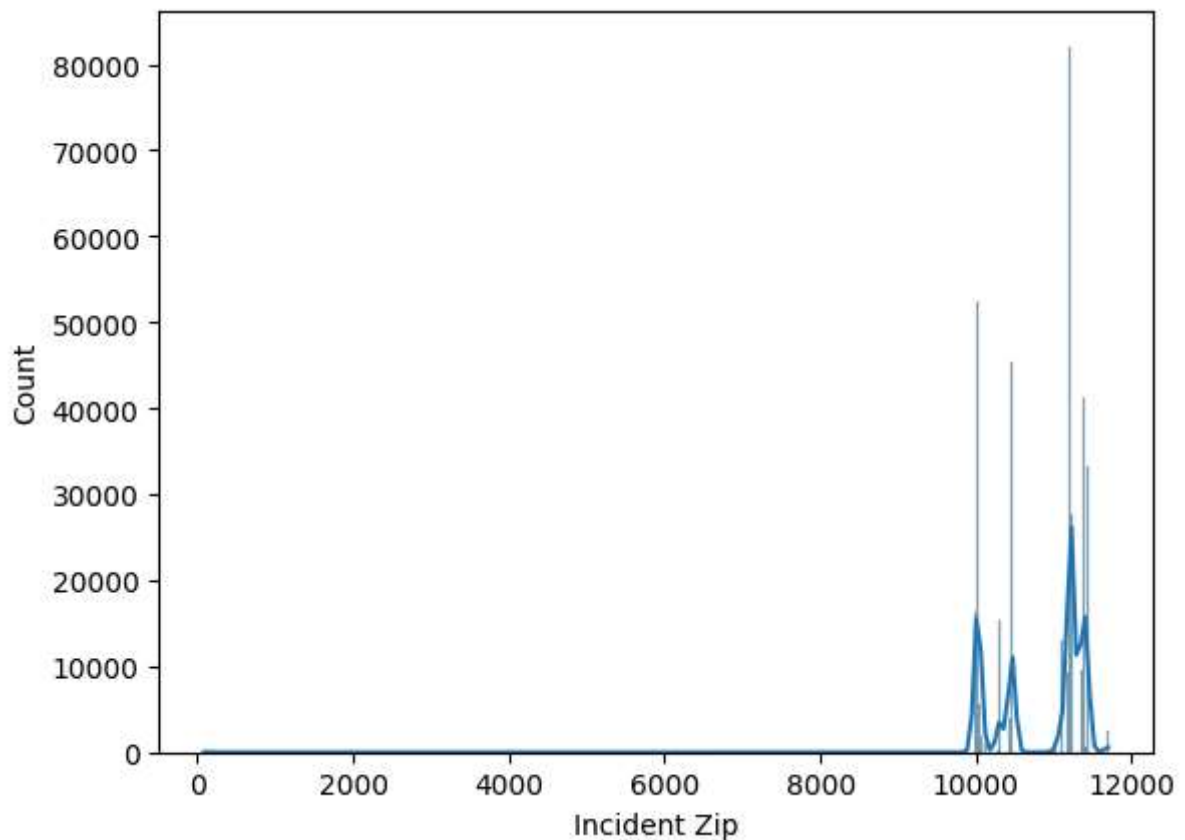
```
Out[23]: count    361560.000000
          mean      10858.496659
          std        578.263114
          min         83.000000
          25%       10314.000000
          50%       11209.000000
          75%       11238.000000
          max       11697.000000
          Name: Incident Zip, dtype: float64
```

```
In [24]: df['Incident Zip'].mode()
```

```
Out[24]: 0    11385.0
          Name: Incident Zip, dtype: float64
```

```
In [25]: sns.histplot(df['Incident Zip'], kde = True)
```

```
plt.show()
```



```
In [26]: sns.distplot(df['Incident Zip'])  
plt.show()
```

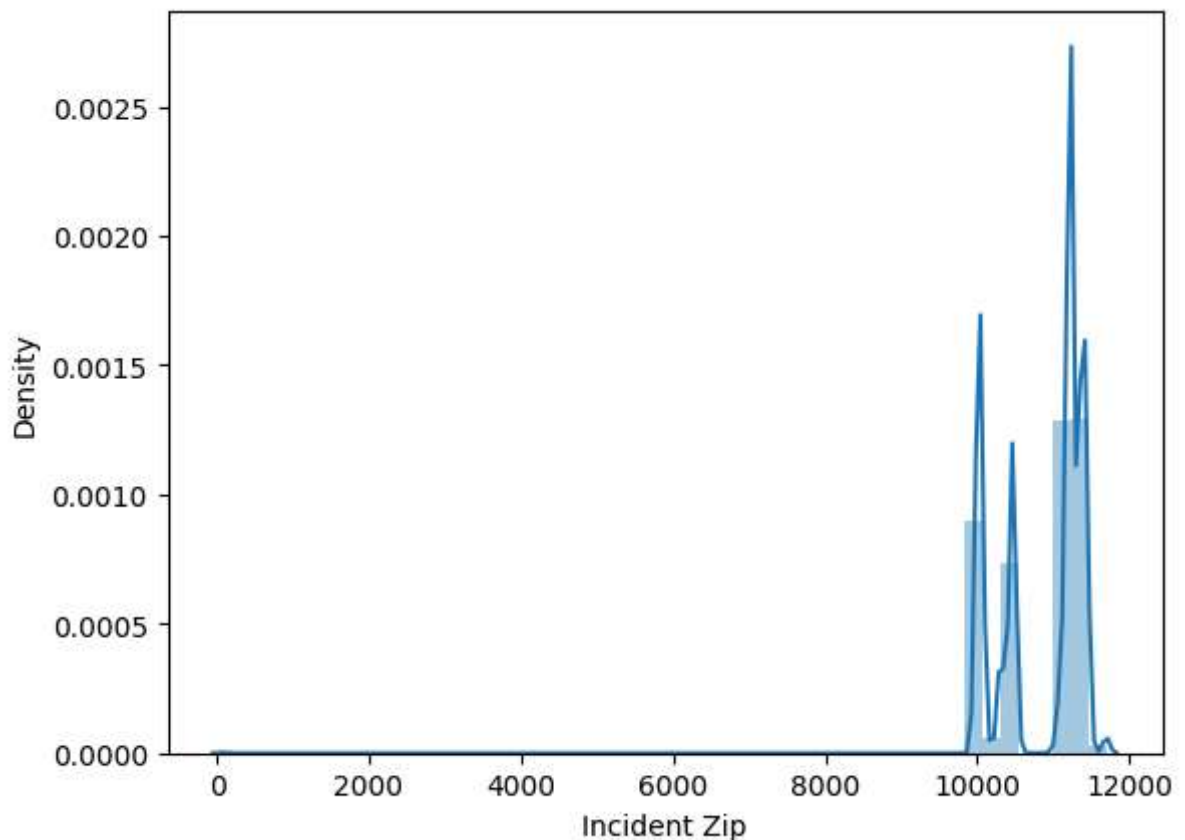
C:\Users\Priyanka Kesarkar\AppData\Local\Temp\ipykernel_788\3996573223.py:1: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(df['Incident Zip'])
```



```
In [27]: df['Incident Zip']=df['Incident Zip'].fillna(df['Incident Zip'].mode()[0])
```

```
In [28]: df['Incident Zip'].isnull().sum()
```

```
Out[28]: np.int64(0)
```

```
In [29]: df['Incident Zip'].value_counts().head()
```

```
Out[29]: Incident Zip
11385.0    9394
11368.0    5384
11234.0    5199
11211.0    4854
11377.0    4525
Name: count, dtype: int64
```

```
In [30]: # Study of address type
df['Address Type'].isnull().sum()
```

```
Out[30]: np.int64(3252)
```

```
In [31]: df['Address Type'].describe()
```

```
Out[31]: count      361306
unique         5
top      ADDRESS
freq      292699
Name: Address Type, dtype: object
```

```
In [32]: df['Address Type'].value_counts()
```

```
Out[32]: Address Type
ADDRESS      292699
INTERSECTION  50514
BLOCKFACE    13786
LATLONG       3931
PLACENAME     376
Name: count, dtype: int64
```

```
In [33]: df['Address Type'].fillna('Unkonown', inplace = True)
```

```
In [34]: df['Address Type'].value_counts()
```

```
Out[34]: Address Type
ADDRESS      292699
INTERSECTION  50514
BLOCKFACE    13786
LATLONG       3931
Unkonown     3252
PLACENAME     376
Name: count, dtype: int64
```

```
In [35]: df['Address Type'].isnull().sum()
```

```
Out[35]: np.int64(0)
```

```
In [36]: # Remove Not Useful Columns of this data
df.drop(['Incident Address', 'Street Name', 'Cross Street 1', 'Cross Street 2', 'In
```

```
In [37]: df.head()
```

Out[37]:

	Unique Key	Created Date	Closed Date	Agency	Complaint Type	Descriptor	Location Type
0	32310363	12/31/2015 11:59:45 PM	01/01/2016 12:55:15 AM	NYPD	Noise - Street/Sidewalk	Loud Music/Party	Street/Sidewalk
1	32309934	12/31/2015 11:59:44 PM	01/01/2016 01:26:57 AM	NYPD	Blocked Driveway	No Access	Street/Sidewalk
2	32309159	12/31/2015 11:59:29 PM	01/01/2016 04:51:03 AM	NYPD	Blocked Driveway	No Access	Street/Sidewalk
3	32305098	12/31/2015 11:57:46 PM	01/01/2016 07:43:13 AM	NYPD	Illegal Parking	Commercial Overnight Parking	Street/Sidewalk
4	32306529	12/31/2015 11:56:58 PM	01/01/2016 03:24:42 AM	NYPD	Illegal Parking	Blocked Sidewalk	Street/Sidewalk

5 rows × 46 columns



```
In [38]: # Filling NA value of city column
df['City'].isnull().sum()
```

Out[38]: np.int64(2997)

```
In [39]: df['City'].describe()
```

Out[39]: count 361561
unique 53
top BROOKLYN
freq 118862
Name: City, dtype: object

```
In [40]: df['City'].value_counts()
```

```

Out[40]: City
          BROOKLYN      118862
          NEW YORK      77312
          BRONX         49171
          STATEN ISLAND 15340
          JAMAICA        8932
          ASTORIA        7991
          FLUSHING       7487
          RIDGEWOOD      6392
          CORONA         5383
          WOODSIDE       4357
          EAST ELMHURST  3558
          OZONE PARK     3446
          ELMHURST       3438
          SOUTH RICHMOND HILL 3431
          MASPETH        3118
          WOODHAVEN      3103
          LONG ISLAND CITY 3028
          SOUTH OZONE PARK 2668
          FRESH MEADOWS  2453
          RICHMOND HILL  2335
          MIDDLE VILLAGE 2291
          QUEENS VILLAGE 2251
          FOREST HILLS   2122
          JACKSON HEIGHTS 2106
          REGO PARK      1807
          BAYSIDE        1550
          COLLEGE POINT  1544
          FAR ROCKAWAY   1397
          WHITESTONE     1369
          HOLLIS         1231
          HOWARD BEACH   1144
          SPRINGFIELD GARDENS 1094
          ROSEDALE       1091
          SAINT ALBANS   1047
          KEW GARDENS    1008
          SUNNYSIDE      944
          Astoria        906
          ROCKAWAY PARK  831
          OAKLAND GARDENS 717
          LITTLE NECK   712
          CAMBRIA HEIGHTS 617
          BELLEROSE      487
          GLEN OAKS      361
          ARVERNE        259
          FLORAL PARK    196
          Long Island City 170
          Woodside       166
          NEW HYDE PARK  129
          CENTRAL PARK   110
          QUEENS         37
          BREEZY POINT   31
          East Elmhurst  30
          Howard Beach   1
          Name: count, dtype: int64

```

```
In [41]: df['City'].fillna('Unknown City', inplace = True)
```

```
In [42]: df['City'].isnull().sum()
```

```
Out[42]: np.int64(0)
```

```
In [43]: df['City'].value_counts()
```

```

Out[43]: City
          BROOKLYN      118862
          NEW YORK      77312
          BRONX         49171
          STATEN ISLAND 15340
          JAMAICA        8932
          ASTORIA        7991
          FLUSHING       7487
          RIDGEWOOD      6392
          CORONA         5383
          WOODSIDE       4357
          EAST ELMHURST  3558
          OZONE PARK     3446
          ELMHURST       3438
          SOUTH RICHMOND HILL 3431
          MASPETH        3118
          WOODHAVEN      3103
          LONG ISLAND CITY 3028
          Unknown City   2997
          SOUTH OZONE PARK 2668
          FRESH MEADOWS  2453
          RICHMOND HILL  2335
          MIDDLE VILLAGE 2291
          QUEENS VILLAGE 2251
          FOREST HILLS   2122
          JACKSON HEIGHTS 2106
          REGO PARK      1807
          BAYSIDE        1550
          COLLEGE POINT  1544
          FAR ROCKAWAY   1397
          WHITESTONE     1369
          HOLLIS         1231
          HOWARD BEACH   1144
          SPRINGFIELD GARDENS 1094
          ROSEDALE       1091
          SAINT ALBANS   1047
          KEW GARDENS    1008
          SUNNYSIDE      944
          Astoria        906
          ROCKAWAY PARK  831
          OAKLAND GARDENS 717
          LITTLE NECK    712
          CAMBRIA HEIGHTS 617
          BELLEROSE      487
          GLEN OAKS      361
          ARVERNE        259
          FLORAL PARK    196
          Long Island City 170
          Woodside       166
          NEW HYDE PARK  129
          CENTRAL PARK   110
          QUEENS         37
          BREEZY POINT   31
          East Elmhurst  30
          Howard Beach    1
          Name: count, dtype: int64

```

```
In [44]: # Remove Landmark Column
        #This Columns was dropped bcoz it had a high number of missing value and was not
        #Why not filling it
        #Landmaark is not imp for analysis and mostly missing filling it would not add va
```

```
In [45]: df['Landmark'].isnull().sum()
```

```
Out[45]: np.int64(364183)
```

```
In [46]: df.drop(['Landmark'], axis = 1, inplace = True)
```

```
In [47]: # Filling NA Value of Facility Type column
```

```
In [48]: df['Facility Type'].isnull().sum()
```

```
Out[48]: np.int64(2389)
```

```
In [49]: df['Facility Type'].describe()
```

```
Out[49]: count      362169
         unique         1
         top      Precinct
         freq      362169
         Name: Facility Type, dtype: object
```

```
In [50]: df['Facility Type'].fillna('Unknown', inplace = True)
```

```
In [51]: df['Facility Type'].value_counts()
```

```
Out[51]: Facility Type
         Precinct      362169
         Unknown       2389
         Name: count, dtype: int64
```

```
In [52]: # Status Column(Not Changes)
         df['Status'].isnull().sum()
```

```
Out[52]: np.int64(0)
```

```
In [53]: # Remove Due Date Column
         df.drop(['Due Date'], inplace = True, axis = 1)
```

```
In [54]: # Resolution Description Column(Not Changes)
         df['Resolution Description'].isnull().sum()
```

```
Out[54]: np.int64(0)
```

```
In [55]: # Study of Resolution Action Updated Date Column
         df['Resolution Action Updated Date'].isnull().sum()
```

```
Out[55]: np.int64(2402)
```

```
In [56]: df['Resolution Action Updated Date'].describe()
```

```
Out[56]: count                362156
         unique                340833
         top      12/19/2015 08:35:28 AM
         freq                   3
         Name: Resolution Action Updated Date, dtype: object
```

```
In [57]: df = df[df['Resolution Action Updated Date'].notnull()]
```

```
In [58]: df['Resolution Action Updated Date'].isnull().sum()
```

```
Out[58]: np.int64(0)
```

```
In [59]: # Remove Borough to Location Columns
         df.columns
```

```
Out[59]: Index(['Unique Key', 'Created Date', 'Closed Date', 'Agency', 'Complaint Type',
               'Descriptor', 'Location Type', 'Incident Zip', 'Address Type', 'City',
               'Facility Type', 'Status', 'Resolution Description',
               'Resolution Action Updated Date', 'Community Board', 'Borough',
               'X Coordinate (State Plane)', 'Y Coordinate (State Plane)',
               'Park Facility Name', 'Park Borough', 'School Name', 'School Number',
               'School Region', 'School Code', 'School Phone Number', 'School Address',
               'School City', 'School State', 'School Zip', 'School Not Found',
               'School or Citywide Complaint', 'Vehicle Type', 'Taxi Company Borough',
               'Taxi Pick Up Location', 'Bridge Highway Name',
               'Bridge Highway Direction', 'Road Ramp', 'Bridge Highway Segment',
               'Garage Lot Name', 'Ferry Direction', 'Ferry Terminal Name', 'Latitude',
               'Longitude', 'Location'],
              dtype='object')
```

```
In [60]: df.drop(['Borough',
                  'X Coordinate (State Plane)', 'Y Coordinate (State Plane)',
                  'Park Facility Name', 'Park Borough', 'School Name', 'School Number',
                  'School Region', 'School Code', 'School Phone Number', 'School Address',
                  'School City', 'School State', 'School Zip', 'School Not Found',
                  'School or Citywide Complaint'], axis = 1, inplace = True)
```

```
In [61]: df.drop(['Vehicle Type', 'Taxi Company Borough',
                  'Taxi Pick Up Location', 'Bridge Highway Name',
                  'Bridge Highway Direction', 'Road Ramp', 'Bridge Highway Segment',
                  'Garage Lot Name', 'Ferry Direction', 'Ferry Terminal Name'], axis = 1, inplace = True)
```

```
In [62]: df['Location'].duplicated().sum()
```

```
Out[62]: np.int64(215424)
```

```
In [63]: df.drop(['Location'], axis = 1, inplace = True)
```

```
In [64]: # Study of Logitude & Latitude Columns
         df[['Latitude', 'Longitude']].isnull().sum()
```

```
Out[64]: Latitude      1708  
Longitude      1708  
dtype: int64
```

```
In [65]: df['Latitude'].describe()
```

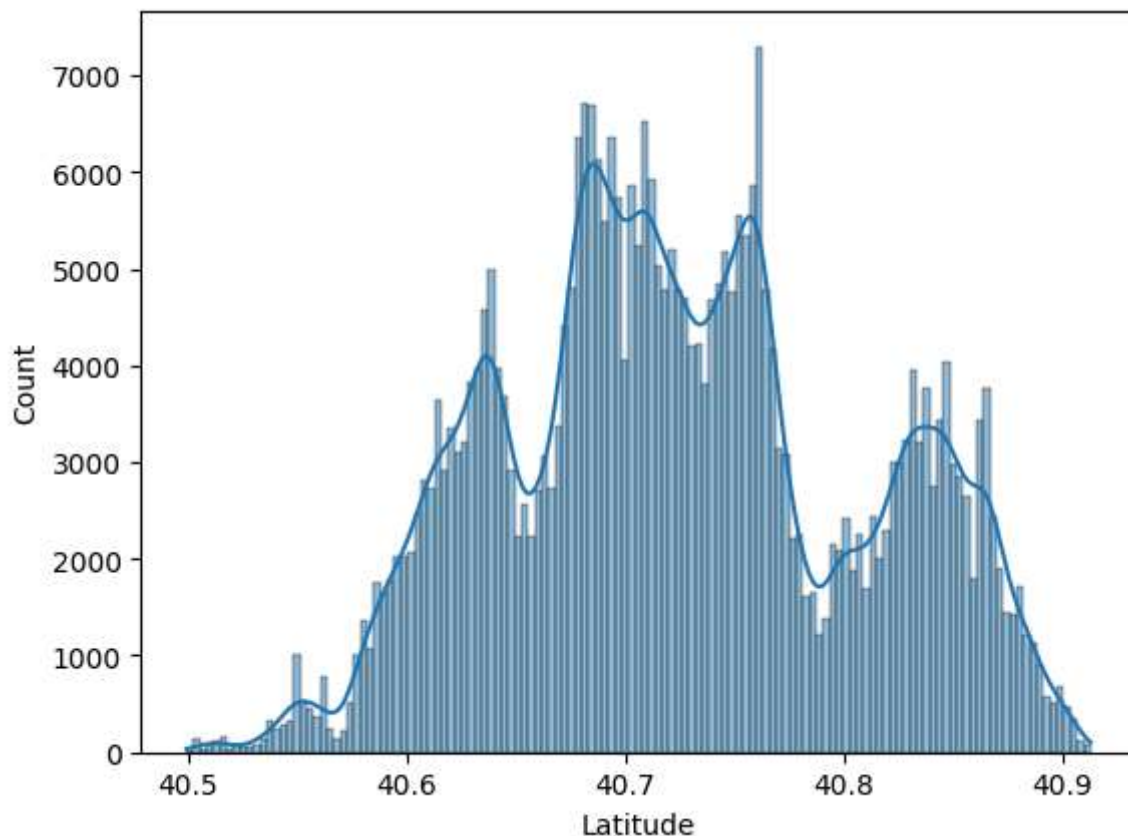
```
Out[65]: count      360448.000000  
mean         40.724980  
std          0.081908  
min          40.499040  
25%          40.668739  
50%          40.718410  
75%          40.778166  
max          40.912869  
Name: Latitude, dtype: float64
```

```
In [66]: df = df[df['Latitude'].notnull()  
df = df[df['Longitude'].notnull()]
```

```
In [67]: df[['Latitude', 'Longitude']].isnull().sum()
```

```
Out[67]: Latitude      0  
Longitude      0  
dtype: int64
```

```
In [68]: sns.histplot(df['Latitude'], kde = True)  
plt.show()
```



```
In [69]: # Final Check
df.columns
```

```
Out[69]: Index(['Unique Key', 'Created Date', 'Closed Date', 'Agency', 'Complaint Type',
               'Descriptor', 'Location Type', 'Incident Zip', 'Address Type', 'City',
               'Facility Type', 'Status', 'Resolution Description',
               'Resolution Action Updated Date', 'Community Board', 'Latitude',
               'Longitude'],
              dtype='object')
```

```
In [70]: df.isnull().sum()
```

```
Out[70]: Unique Key          0
         Created Date        0
         Closed Date        17
         Agency             0
         Complaint Type      0
         Descriptor         0
         Location Type       0
         Incident Zip        0
         Address Type        0
         City               0
         Facility Type       0
         Status             0
         Resolution Description 0
         Resolution Action Updated Date 0
         Community Board     0
         Latitude           0
         Longitude          0
         dtype: int64
```

```
In [71]: df.shape
```

```
Out[71]: (360448, 17)
```

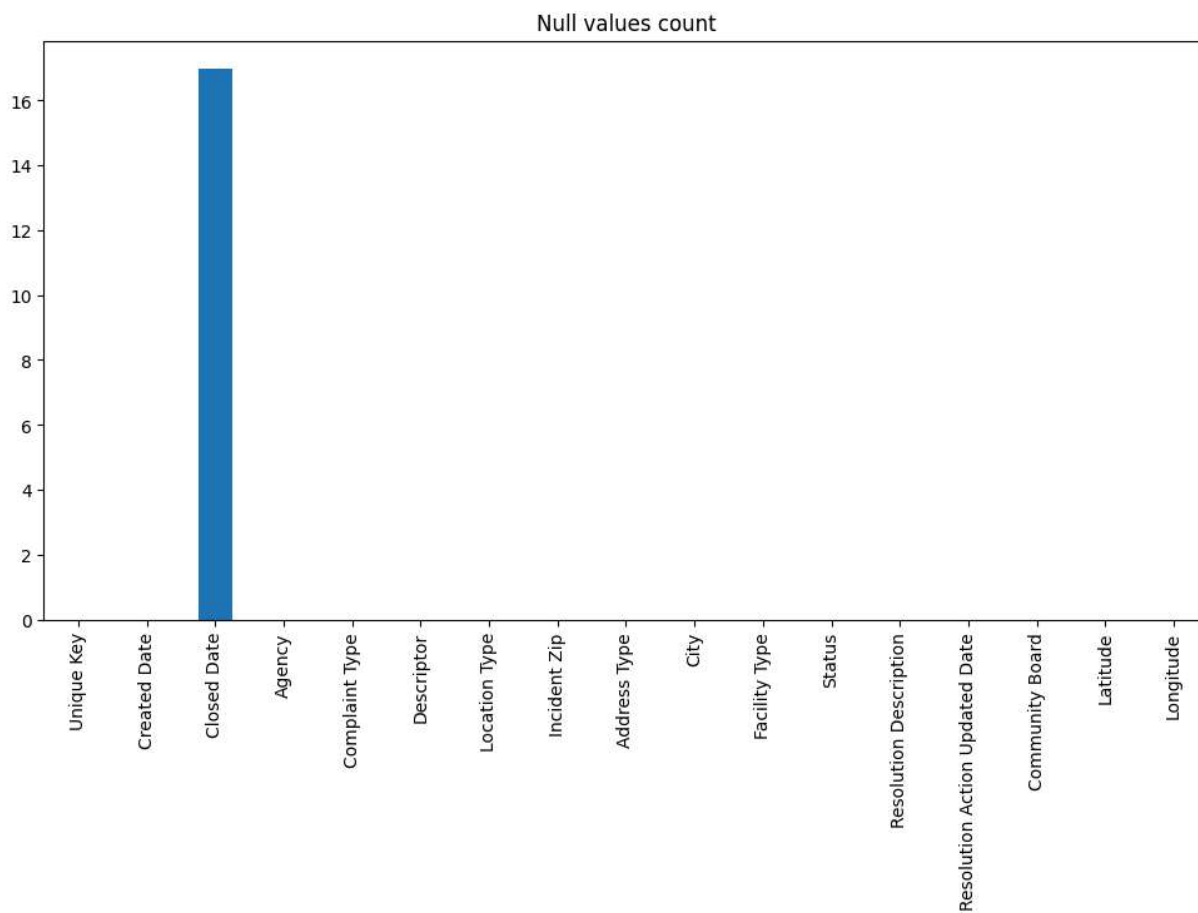
```
In [72]: df.head()
```

Out[72]:

	Unique Key	Created Date	Closed Date	Agency	Complaint Type	Descriptor	Location Type
0	32310363	12/31/2015 11:59:45 PM	01/01/2016 12:55:15 AM	NYPD	Noise - Street/Sidewalk	Loud Music/Party	Street/Sidewalk
1	32309934	12/31/2015 11:59:44 PM	01/01/2016 01:26:57 AM	NYPD	Blocked Driveway	No Access	Street/Sidewalk
2	32309159	12/31/2015 11:59:29 PM	01/01/2016 04:51:03 AM	NYPD	Blocked Driveway	No Access	Street/Sidewalk
3	32305098	12/31/2015 11:57:46 PM	01/01/2016 07:43:13 AM	NYPD	Illegal Parking	Commercial Overnight Parking	Street/Sidewalk
4	32306529	12/31/2015 11:56:58 PM	01/01/2016 03:24:42 AM	NYPD	Illegal Parking	Blocked Sidewalk	Street/Sidewalk

Step 2: Exploratory Data Analysis

```
In [73]: # Frequency Plot of null values
df.isnull().sum().plot(kind = 'bar', figsize = (12,6))
plt.title('Null values count')
plt.show()
```



```
In [74]: # Missing Value Treatments  
# Removes rows where closed date is null
```

```
In [75]: df = df[df['Closed Date'].notnull()]
```

```
In [76]: df['Closed Date'].isnull().sum()
```

```
Out[76]: np.int64(0)
```

```
In [77]: df.isnull().sum()
```

```
Out[77]: Unique Key          0
         Created Date       0
         Closed Date        0
         Agency             0
         Complaint Type     0
         Descriptor         0
         Location Type      0
         Incident Zip       0
         Address Type       0
         City               0
         Facility Type      0
         Status             0
         Resolution Description 0
         Resolution Action Updated Date 0
         Community Board    0
         Latitude           0
         Longitude          0
         dtype: int64
```

```
In [78]: # Analyze the date column
df = df.copy()
df['Created Date']=pd.to_datetime(df['Created Date'], errors = 'coerce')
df['Closed Date']=pd.to_datetime(df['Closed Date'], errors = 'coerce')
```

C:\Users\Priyanka Kesarkar\AppData\Local\Temp\ipykernel_788\1490084153.py:3: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.

```
df['Created Date']=pd.to_datetime(df['Created Date'], errors = 'coerce')
```

C:\Users\Priyanka Kesarkar\AppData\Local\Temp\ipykernel_788\1490084153.py:4: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.

```
df['Closed Date']=pd.to_datetime(df['Closed Date'], errors = 'coerce')
```

```
In [79]: df[['Created Date', 'Closed Date']].dtypes
```

```
Out[79]: Created Date    datetime64[ns]
         Closed Date    datetime64[ns]
         dtype: object
```

```
In [80]: df[['Created Date', 'Closed Date']].head()
```

```
Out[80]:
```

	Created Date	Closed Date
0	2015-12-31 23:59:45	2016-01-01 00:55:15
1	2015-12-31 23:59:44	2016-01-01 01:26:57
2	2015-12-31 23:59:29	2016-01-01 04:51:03
3	2015-12-31 23:57:46	2016-01-01 07:43:13
4	2015-12-31 23:56:58	2016-01-01 03:24:42

```
In [81]: # Remove Incorrect Timeline
df = df[df['Closed Date'] >= df['Created Date']]
```

```
In [82]: df[['Created Date', 'Closed Date']].head()
```

```
Out[82]:
```

	Created Date	Closed Date
0	2015-12-31 23:59:45	2016-01-01 00:55:15
1	2015-12-31 23:59:44	2016-01-01 01:26:57
2	2015-12-31 23:59:29	2016-01-01 04:51:03
3	2015-12-31 23:57:46	2016-01-01 07:43:13
4	2015-12-31 23:56:58	2016-01-01 03:24:42

```
In [83]: # Calculate Time Elapsed in Closed & Creation Date
df['Request Closing Time'] = df['Closed Date'] - df['Created Date']
```

```
In [84]: df.shape
```

```
Out[84]: (360431, 18)
```

```
In [85]: df.columns
```

```
Out[85]: Index(['Unique Key', 'Created Date', 'Closed Date', 'Agency', 'Complaint Type',
              'Descriptor', 'Location Type', 'Incident Zip', 'Address Type', 'City',
              'Facility Type', 'Status', 'Resolution Description',
              'Resolution Action Updated Date', 'Community Board', 'Latitude',
              'Longitude', 'Request Closing Time'],
              dtype='object')
```

```
In [86]: # Convert Calculated Date to Seconds
df['Request Closing Time'] = df['Request Closing Time'].dt.total_seconds()
```

```
In [87]: df['Request Closing Time'].dtype
```

```
Out[87]: dtype('float64')
```

```
In [88]: # View descriptive statistics for the newly created column
df['Request Closing Time'].describe()
```

```
Out[88]: count    3.604310e+05
mean      1.508083e+04
std       2.097421e+04
min       6.100000e+01
25%       4.529000e+03
50%       9.604000e+03
75%       1.884100e+04
max       2.134342e+06
Name: Request Closing Time, dtype: float64
```

```
In [89]: # Check null values of Complaint Type & City  
df[['Complaint Type', 'City']].isnull().sum()
```

```
Out[89]: Complaint Type    0  
        City              0  
        dtype: int64
```

```
In [90]: # Filling NA Value with unknown city in city column  
# df['City'].fillna('Unknown City, inplace = True')
```

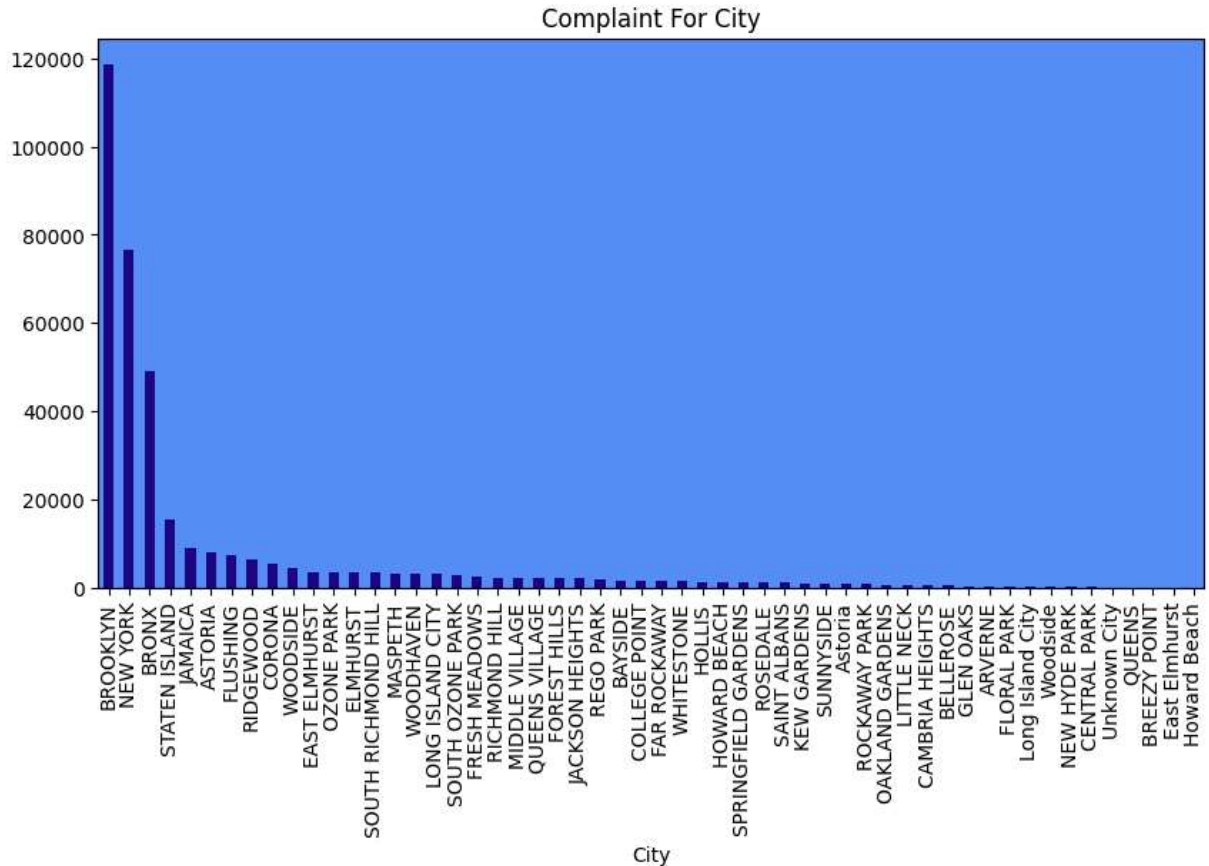
```
In [91]: df['City'].value_counts()
```

```

Out[91]: City
          BROOKLYN      118624
          NEW YORK      76626
          BRONX         49045
          STATEN ISLAND 15323
          JAMAICA       8911
          ASTORIA       7974
          FLUSHING      7481
          RIDGEWOOD     6388
          CORONA        5381
          WOODSIDE      4353
          EAST ELMHURST 3557
          OZONE PARK    3446
          ELMHURST      3437
          SOUTH RICHMOND HILL 3430
          MASPETH       3116
          WOODHAVEN     3101
          LONG ISLAND CITY 3018
          SOUTH OZONE PARK 2668
          FRESH MEADOWS 2447
          RICHMOND HILL 2333
          MIDDLE VILLAGE 2290
          QUEENS VILLAGE 2251
          FOREST HILLS  2120
          JACKSON HEIGHTS 2105
          REGO PARK     1805
          BAYSIDE       1548
          COLLEGE POINT 1544
          FAR ROCKAWAY  1396
          WHITESTONE    1367
          HOLLIS        1231
          HOWARD BEACH  1143
          SPRINGFIELD GARDENS 1093
          ROSEDALE      1086
          SAINT ALBANS  1047
          KEW GARDENS   1008
          SUNNYSIDE     944
          Astoria       905
          ROCKAWAY PARK 829
          OAKLAND GARDENS 715
          LITTLE NECK  712
          CAMBRIA HEIGHTS 617
          BELLEROSE     487
          GLEN OAKS     361
          ARVERNE       258
          FLORAL PARK   196
          Long Island City 170
          Woodside      166
          NEW HYDE PARK 129
          CENTRAL PARK  110
          Unknown City  41
          QUEENS        36
          BREEZY POINT  31
          East Elmhurst 30
          Howard Beach  1
          Name: count, dtype: int64

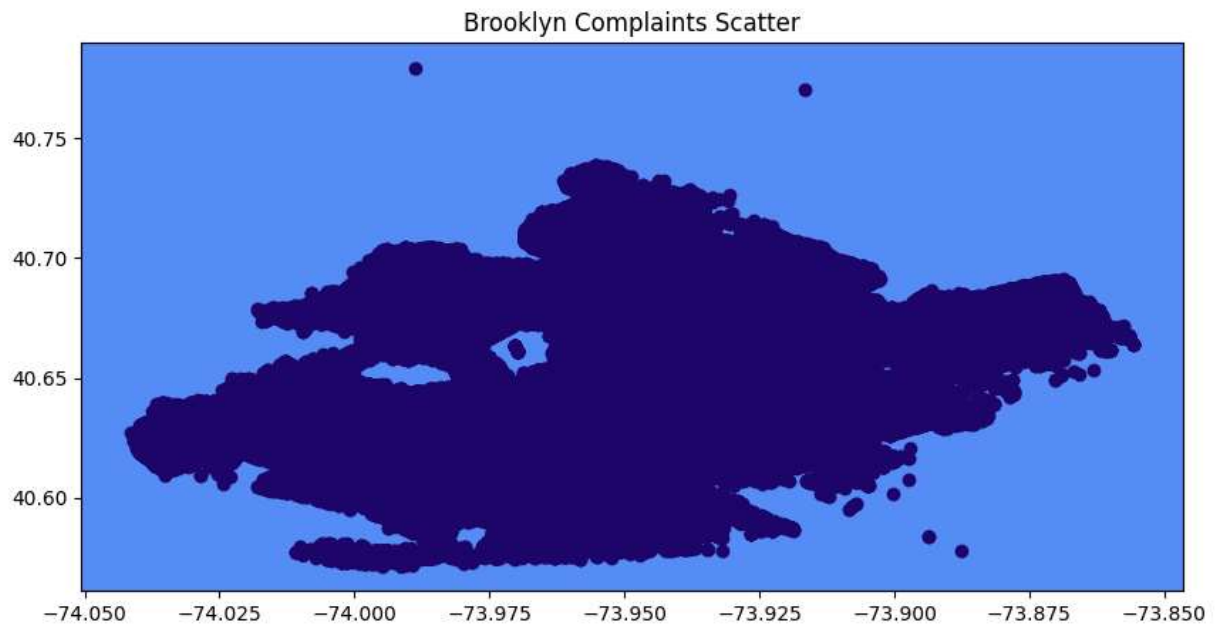
```

```
In [92]: # Draw Frequency plot for the complaint in each city
plt.figure(figsize = (10,5))
ax = df['City'].value_counts().plot(kind = 'bar', color = '#1f0585')
ax.set_facecolor('#588cf5')
plt.title('Complaint For City')
plt.show()
```



```
In [93]: # Create a Scatter & Hexbin Plot of the concentration of complaints Across Brooklyn
```

```
In [94]: plt.figure(figsize = (10,5))
brooklyn = df[df['City']== "BROOKLYN"]
plt.scatter(x = brooklyn['Longitude'], y = brooklyn['Latitude'], color = '#22056b')
plt.gca().set_facecolor('#588cf5')
plt.title('Brooklyn Complaints Scatter')
plt.show()
plt.figure(figsize = (10,5))
ax = brooklyn.plot.hexbin(x = 'Longitude', y = 'Latitude', gridsize = 25, cmap = 'Bl')
ax.set_facecolor('#1f0585')
plt.title('Brooklyn Complaints Hexbin')
plt.show()
```

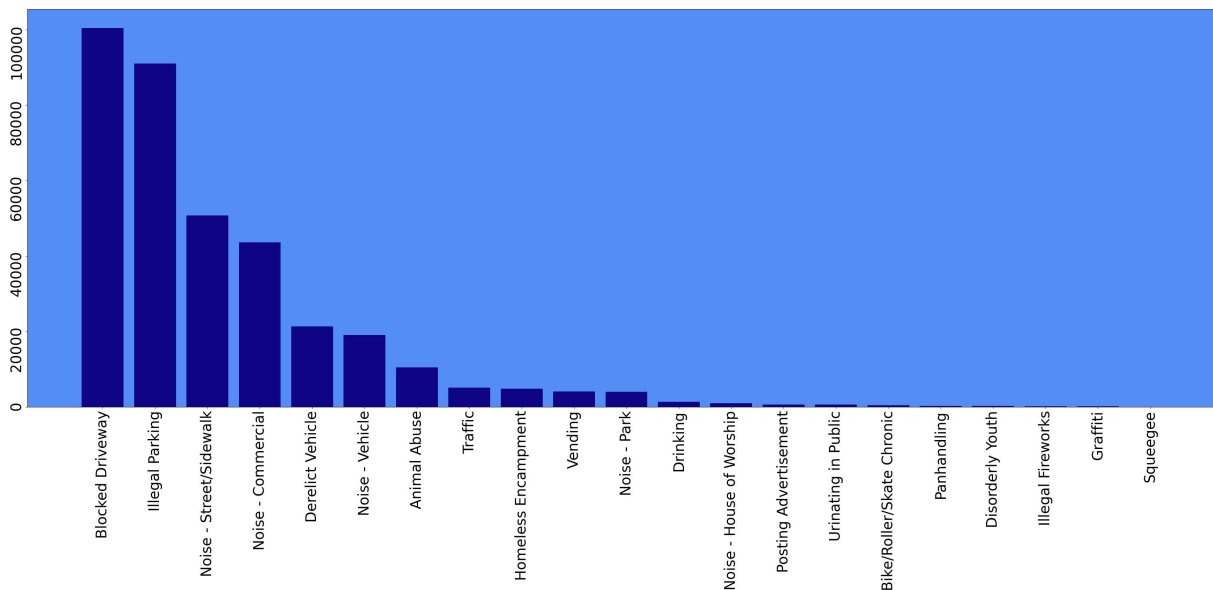


Step 3 - Major Complaints Types

```
In [95]: # Plot a Bar Graph to show the Type of Complaint
com_type = df['Complaint Type'].value_counts()
print(type(com_type))
print(com_type)
```

```
<class 'pandas.core.series.Series'>
Complaint Type
Blocked Driveway          100487
Illegal Parking           91084
Noise - Street/Sidewalk   50815
Noise - Commercial        43638
Derelict Vehicle          21425
Noise - Vehicle           19124
Animal Abuse              10503
Traffic                   5166
Homeless Encampment       4828
Vending                   4162
Noise - Park              3994
Drinking                  1400
Noise - House of Worship  1061
Posting Advertisement      678
Urinating in Public        641
Bike/Roller/Skate Chronic  463
Panhandling                320
Disorderly Youth           314
Illegal Fireworks          167
Graffiti                  157
Squeegee                   4
Name: count, dtype: int64
```

```
In [96]: plt.figure(figsize = (45,15))
plt.bar(x = com_type.index, height = com_type, color = '#100987')
plt.xticks(rotation = 90, size = 30)
plt.yticks(rotation = 90, size = 30)
plt.gca().set_facecolor('#588cf5')
plt.show()
```



```
In [97]: # Check the frequency of various types of complaint for new york city
nyc = df[df['City'] == 'NEW YORK']
nyc['Complaint Type'].value_counts()
```

```
Out[97]: Complaint Type
Noise - Street/Sidewalk    22079
Noise - Commercial         18668
Illegal Parking            14366
Noise - Vehicle            6179
Homeless Encampment        3020
Blocked Driveway           2687
Vending                    2619
Animal Abuse               1926
Traffic                    1749
Noise - Park               1200
Derelict Vehicle           688
Drinking                   320
Urinating in Public        264
Bike/Roller/Skate Chronic  249
Noise - House of Worship   217
Panhandling                203
Disorderly Youth           80
Posting Advertisement       49
Illegal Fireworks           34
Graffiti                   25
Squeegee                    4
Name: count, dtype: int64
```

```
In [98]: # Top 10 Complaints Types
df['Complaint Type'].value_counts().head(10)
```

```
Out[98]: Complaint Type
Blocked Driveway    100487
Illegal Parking     91084
Noise - Street/Sidewalk 50815
Noise - Commercial  43638
Derelict Vehicle    21425
Noise - Vehicle     19124
Animal Abuse        10503
Traffic             5166
Homeless Encampment  4828
Vending             4162
Name: count, dtype: int64
```

```
In [99]: # Display the various type of Complaints in each city
df.groupby('City')['Complaint Type'].value_counts()
```

```
Out[99]: City      Complaint Type
ARVERNE  Illegal Parking      62
         Blocked Driveway    50
         Animal Abuse        46
         Derelict Vehicle    32
         Noise - Street/Sidewalk 29
         ...
Woodside Illegal Parking      124
         Blocked Driveway    27
         Derelict Vehicle     8
         Noise - Street/Sidewalk 5
         Noise - Commercial   2
Name: count, Length: 782, dtype: int64
```

```
In [100... # Create a Dataframe, df_new(Create Pivot Table)
df_new = pd.crosstab(df['Complaint Type'], df['City'])
df_new.head()
```

Out[100...

City	ARVERNE	ASTORIA	Astoria	BAYSIDE	BELLEROSE	BREEZY POINT	BRONX	BI
Complaint Type								
Animal Abuse	46	170	0	53	15	2	1967	
Bike/Roller/Skate Chronic	0	16	0	0	1	0	22	
Blocked Driveway	50	3436	159	514	138	3	17051	
Derelict Vehicle	32	426	14	231	120	3	2399	
Disorderly Youth	2	5	0	2	2	0	66	

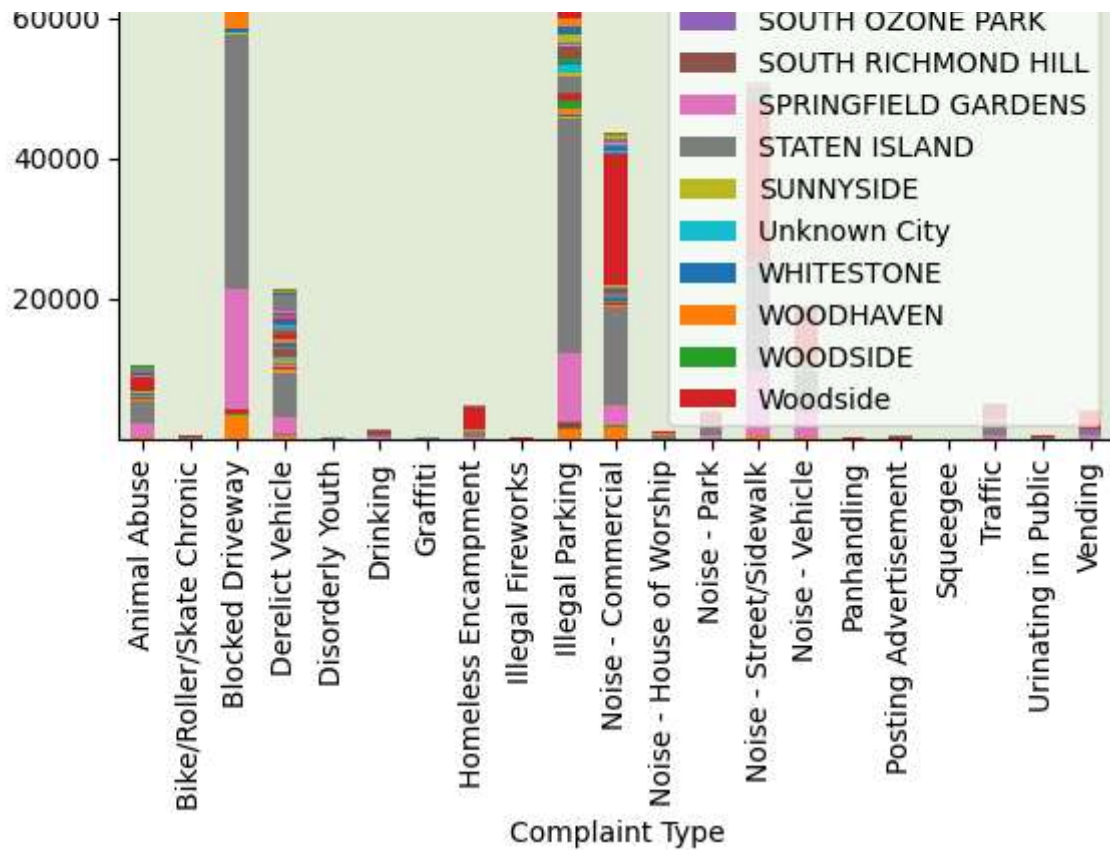
5 rows × 54 columns

Step 4: Visualization

```
In [101... # Multi City Complaint Chart
plt.figure(figsize = (10, 5))
ax = df_new.plot(kind = 'bar', stacked = True)
ax.set_facecolor('#E2EFD9')
plt.show()
```

<Figure size 1000x500 with 0 Axes>



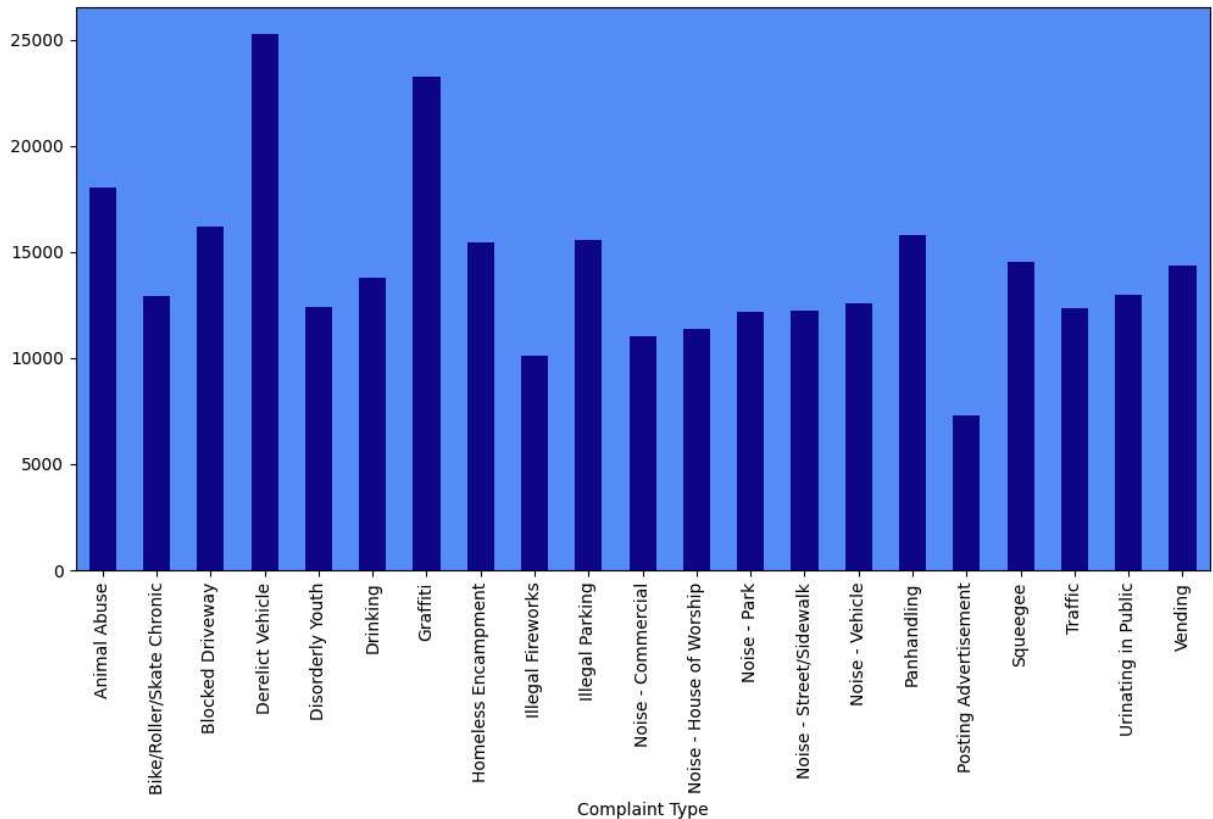


```
In [102... # Sort by average Request Closing Timegrouping
df.groupby('Complaint Type')['Request Closing Time'].mean().sort_values()
```

```
Out[102... Complaint Type
Posting Advertisement      7287.171091
Illegal Fireworks         10122.347305
Noise - Commercial        11040.838260
Noise - House of Worship  11398.217719
Noise - Park              12196.250125
Noise - Street/Sidewalk   12218.206435
Traffic                   12326.666280
Disorderly Youth         12395.754777
Noise - Vehicle           12586.443840
Bike/Roller/Skate Chronic 12941.995680
Urinating in Public       12959.293292
Drinking                  13803.985000
Vending                   14355.265738
Squeegee                  14560.250000
Homeless Encampment      15479.961268
Illegal Parking           15583.577774
Panhandling               15822.803125
Blocked Driveway         16217.725089
Animal Abuse              18051.110921
Graffiti                 23276.343949
Derelict Vehicle          25281.671972
Name: Request Closing Time, dtype: float64
```

Step 5: Average Response Time

```
In [103... # Visualize the average of request closing time
plt.figure(figsize = (12,6))
ax = df.groupby('Complaint Type')['Request Closing Time'].mean().plot(kind = 'bar',
ax.set_facecolor('#588cf5')
plt.show()
```



Step 6: Statistical Analysis using P-Values

```
In [104... from scipy.stats import f_oneway
groups = [group['Request Closing Time'].values
           for name, group in df.groupby('Complaint Type')]
f_stat, p_value = f_oneway(*groups)
print(p_value)
```

0.0

Step 7 : Kruskal-Wallis Test

```
In [105... from scipy.stats import kruskal

groups = [group['Request Closing Time'].values
           for name, group in df.groupby('Complaint Type')]
```

```
stat, p = kruskal(*groups)

print(p)
```

0.0

Step 8: Present Observation

```
In [106...  # Missing data handled successfully
            # Most complaints come from specific cities
            # Some complaint types take longer time
            # Response time varies Significantly
```

```
In [107... df.to_csv('Customer Service Analysis.csv', index = False)
            print('Saved Successfully')
```

Saved Successfully

In []:

In []: